

Capa de datos distribuida y pipeline analítico paralelo para el Sistema de Control de Laboratorios e Informática (SCLI)

Entrega 3 del Proyecto Fin de Curso

Freddy Farinango, Iván Villamarín, Isaías Urbina, Harold Vinuesa

Equipo PFC ForáCode

Carrera de Ingeniería de Software, Facultad de Ciencias de la Computación

Universidad Técnica Estatal de Quevedo (UTEQ), Ecuador

https://github.com/iavillamarin98-pred/Proyecto_Fin_de_Curso_scli

Resumen

Este informe documenta la Entrega 3 (E3) del Proyecto Fin de Curso del sistema SCLI (Sistema de Control de Laboratorios e Informática), centrada en la capa de persistencia distribuida y el pipeline analítico paralelo que completan la arquitectura de microservicios construida en entregas previas. Se diseñó e implementó un esquema de fragmentación horizontal temporal y replicación triple sobre un clúster CockroachDB de tres nodos dedicado al microservicio de reservas y solicitudes, documentado mediante dos registros de decisión arquitectónica (ADR). Se verificó experimentalmente la tolerancia a fallos del clúster ante la caída de uno y de dos nodos, observando disponibilidad continua con quórum de dos réplicas y pérdida de disponibilidad al perder el quórum, consistente con las garantías teóricas del algoritmo de consenso Raft. Se implementó además un pipeline de procesamiento paralelo en PySpark con cinco transformaciones sobre el esquema de reservas y una línea base equivalente en pandas, ambos funcionalmente completos pero, a la fecha de este informe, sin ejecuciones registradas del protocolo comparativo. El informe documenta con honestidad tanto lo alcanzado como lo pendiente, y traza la ruta de integración con la Entrega 4.

Resumen

Abstract. This report documents the third deliverable (E3) of the capstone project for SCLI (Laboratory and Computing Resources Control System), focused on the distributed persistence layer and the parallel analytics pipeline that complete the microservice architecture built in previous deliverables. A temporal horizontal-fragmentation and triple-replication schema was designed and implemented over a three-node CockroachDB cluster dedicated to the reservations microservice, documented through two architectural decision records (ADR). Cluster fault tolerance was experimentally verified under single- and double-node failures, observing continued availability with a two-replica quorum and unavailability upon quorum loss, consistent with the theoretical guarantees of the Raft consensus algorithm. A parallel processing pipeline with five transformations was additionally implemented in PySpark over the reservations schema, along with a functionally equivalent pandas baseline; both are complete but, as of this report, without

recorded executions of the comparative protocol. The report documents both achievements and pending work honestly, and outlines the integration path with the fourth deliverable.

1. Introducción

El proyecto SCLI (Sistema de Control de Laboratorios e Informática) atiende la gestión de laboratorios, reservas y solicitudes académicas de la Universidad Técnica Estatal de Quevedo. El sistema se construye de manera acumulativa a lo largo del Proyecto Fin de Curso: las entregas previas establecieron el dominio, los actores y la arquitectura de microservicios desplegada mediante contenedores, con un servicio de autenticación, un servicio de usuarios, un servicio académico-laboratorios y un servicio de reservas y solicitudes, cada uno propietario de su propia base de datos, un *api-gateway* que centraliza el enrutamiento hacia los cuatro servicios, y autenticación basada en JSON Web Tokens (JWT) según la especificación RFC 7519 [1], validados por cada microservicio de forma independiente sin necesidad de consultar al servicio de autenticación en cada petición. Esta arquitectura de microservicios sigue los principios de sistemas orientados a recursos discutidos por Fielding [2] en su caracterización del estilo REST, y los límites de servicio se trazaron siguiendo las heurísticas de descomposición por dominio de Newman [2].

La Entrega 3 (E3) añade las dos capas que hacían falta para que el sistema sea distribuido en sentido pleno: una capa de persistencia distribuida real, con fragmentación, replicación y consenso, sustituyendo las bases mono-nodo utilizadas hasta ahora, y un pipeline de procesamiento paralelo que consume esos datos y mide su rendimiento frente a un baseline secuencial. Concretamente, este informe documenta:

- el diseño del esquema de fragmentación horizontal temporal para las tablas del dominio de reservas, registrado en el ADR-003;
- el despliegue de un clúster CockroachDB de tres nodos dedicado al microservicio de reservas y solicitudes, con una política de replicación triple documentada en el ADR-004;
- la verificación experimental de la tolerancia a fallos del clúster ante la caída de uno y de dos nodos, con evidencia de latencia y capturas de consola;

- la integración de la capa de aplicación con el nivel de aislamiento transaccional serializable que exige un sistema consistente como CockroachDB;
- el pipeline analítico en PySpark, con cinco transformaciones sobre el esquema de reservas, y su línea base equivalente en pandas.

Un principio explícito de esta entrega, enunciado en la guía docente, es la complementariedad con la Entrega 4: cada artefacto producido aquí –esquema, clúster, pipeline y protocolo experimental– se plantea como una pieza reutilizable, no como un ejercicio desechable, condición necesaria para que la E4 aborde en el siguiente ciclo la calidad, la interfaz web, la interfaz móvil y la observabilidad de manera integral. Por esa misma razón, este informe distingue con cuidado entre lo que el equipo verificó empíricamente y lo que quedó diseñado o codificado pero todavía sin ejecutar, en lugar de presentar resultados que no fueron efectivamente medidos. El resto del documento se organiza siguiendo esa misma lógica: la Sección 2 sitúa el trabajo frente a la literatura; la Sección 3 formaliza el marco teórico aplicado; las Secciones 4 a 6 documentan lo construido; la Sección 7 documenta el protocolo de medición diseñado y su estado de ejecución; y las Secciones 8 y 9 interpretan los resultados y trazan la ruta hacia la siguiente entrega.

2. Trabajos relacionados

El diseño de bases de datos distribuidas parte de los principios de transparencia de fragmentación, replicación y ubicación establecidos por Özsu y Valduriez [3], quienes caracterizan un sistema de base de datos distribuido como un conjunto de bases lógicamente relacionadas que se perciben como una única base ante el usuario. Kleppmann [4] complementa esta perspectiva con un tratamiento orientado a la ingeniería de sistemas de datos modernos, discutiendo de forma extensa los compromisos entre consistencia, disponibilidad y particionamiento que enmarcan decisiones como las tomadas en este proyecto.

El teorema CAP de Brewer [5] y su reformulación posterior como PACELC por Abadi [5] son el marco teórico bajo el cual se interpreta el comportamiento observado del clúster: CockroachDB se posiciona explícitamente como un sistema CP, y el trabajo de Taft *et al.* [6] –el artículo de referencia de sus autores originales publicado en SIGMOD 2020– describe en detalle cómo la base de datos logra consistencia serializable mediante el algoritmo de consenso Raft de Ongaro y Ousterhout [7], sacrificando disponibilidad ante particiones de red severas. La verificación de tolerancia a fallos realizada en este trabajo (Sección 5) reproduce, a pequeña escala y sobre datos propios, el comportamiento que estos trabajos predicen teóricamente.

En la capa de procesamiento paralelo, el modelo de *Resilient Distributed Datasets* (RDD) de Zaharia *et al.* [8] fundamenta el diseño del pipeline PySpark implementado en este proyecto, y la guía de Chambers y Zaharia [8] orienta las decisiones prácticas de la API de DataFrames utilizada. La re-

visión de Salloum *et al.* [8] sobre analítica de macrodatos con Apache Spark documenta patrones de uso comparables a los aplicados aquí (filtrado, combinación de conjuntos, ventanas de agregación y transformaciones de aprendizaje automático ligero). El marco metodológico para reportar la comparación de rendimiento se apoya en las guías de experimentación en ingeniería de software de Wohlin *et al.* [9] y en las recomendaciones de Jain [10] para el diseño experimental de sistemas de cómputo, si bien –como se explica en la Sección 7– la ejecución completa de ese protocolo queda pendiente para un ciclo posterior.

Cooper *et al.* [11] introducen el benchmark YCSB (*Yahoo! Cloud Serving Benchmark*), un marco de referencia ampliamente adoptado para comparar sistemas de almacenamiento distribuido bajo cargas mixtas de lectura y escritura; aunque este proyecto no ejecuta YCSB directamente, su enfoque de variar sistemáticamente la carga y reportar percentiles de latencia inspira la estructura de la Tabla 2 de este informe y es una referencia natural para extender el protocolo experimental de la Sección 7 en la siguiente entrega. Finalmente, la práctica de documentar decisiones arquitectónicas mediante registros ligeros (*Architectural Decision Records*, ADR) –aplicada en los ADR-003 y ADR-004 de la Sección 4– sigue la convención popularizada por Michael Nygard y adoptada de forma extendida en proyectos de arquitectura de microservicios como el descrito por Newman [2], cuya guía sobre sistemas de grano fino orientó también las decisiones de límites de servicio tomadas en las entregas previas del PFC.

3. Fundamento teórico

3.1. Modelo de bases de datos distribuidas

Un sistema de base de datos distribuido (SBDD) se define como un conjunto de múltiples bases de datos lógicamente relacionadas, gestionadas por un único sistema y percibidas por el usuario como una única base [3]. Sus objetivos de diseño son la transparencia de fragmentación, replicación y ubicación, la autonomía local de cada nodo, la disponibilidad y la escalabilidad horizontal.

La *fragmentación horizontal* de una relación R produce fragmentos $R_1, R_2, \dots, R_n$ mediante selección sobre un atributo particionador:

$$R_i = \sigma_{p_i}(R) \quad (1)$$

donde $\{p_i\}$ son predicados mutuamente excluyentes y colectivamente exhaustivos. La *replicación* exige un factor $f \geq 3$ para tolerar la caída de un nodo conservando quórum mayoritario: un sistema con factor de replicación f tolera la pérdida simultánea de hasta $\lfloor (f-1)/2 \rfloor$ réplicas por rango, siempre que se mantenga el quórum de escritura $\lceil f/2 \rceil + 1$ [7, 6]. Con $f = 3$, el quórum de escritura es 2 y el sistema tolera la caída de $\lfloor 2/2 \rfloor = 1$ réplica, exactamente el comportamiento verificado empíricamente en la Sección 5.

3.1.1. Taxonomía de transparencia

Özsu y Valduriez [3] descomponen la transparencia de un SBDD en cuatro niveles independientes, útiles para situar con precisión qué garantiza efectivamente la arquitectura de esta entrega. La *transparencia de fragmentación* oculta al cliente que una relación está dividida en fragmentos; en SCLI, la aplicación consulta `solicitudes_reserva` como una tabla única sin conocer, ni deber conocer, sus futuras particiones temporales. La *transparencia de ubicación* oculta en qué nodo físico reside cada fragmento; CockroachDB la provee de forma nativa mediante su capa de distribución, que enruta cada consulta SQL al conjunto de rangos correspondiente sin intervención de la aplicación. La *transparencia de replicación* oculta cuántas copias existen de cada dato y cómo se sincronizan; en este proyecto corresponde exactamente al mecanismo de Raft descrito en la Sección 3.3. Finalmente, la *transparencia de concurrencia y de fallos* garantiza que transacciones simultáneas y fallos parciales de nodos no produzcan resultados incorrectos ni requieran lógica especial en la capa de aplicación; esta es, en sentido estricto, la propiedad que la Sección 5.7 verifica empíricamente para el caso de la caída de un nodo.

3.2. Teorema CAP y modelo PACELC

El teorema CAP establece que un sistema distribuido no puede satisfacer simultáneamente consistencia, disponibilidad y tolerancia a particiones de red [5]. El modelo PACELC extiende esta discusión incorporando el compromiso entre latencia y consistencia incluso en ausencia de particiones: en presencia de una partición (P), el sistema elige entre disponibilidad (A) y consistencia (C); en su ausencia (E), elige entre latencia (L) y consistencia (C) [5]. CockroachDB se ubica como un sistema PC/EC: prioriza consistencia serializable tanto ante particiones como en operación normal, sacrificando disponibilidad de escritura cuando el clúster pierde el quórum mayoritario [6]. Este posicionamiento es exactamente el que se observó en el escenario de dos nodos caídos descrito en la Sección 5.7.

3.3. Consenso distribuido: Raft

El algoritmo Raft descompone el problema del consenso distribuido en tres subproblemas independientes: elección de líder, replicación del registro de operaciones (*log*) y seguridad [7]. En CockroachDB, cada rango de datos –la unidad atómica de fragmentación y replicación del motor– constituye un grupo Raft independiente con su propio líder. Una escritura se considera comprometida (*committed*) cuando el líder recibe confirmación de una mayoría estricta de las réplicas del grupo, garantizando durabilidad frente a la caída de una minoría de nodos. Cuando el líder deja de estar disponible y sobreviven réplicas suficientes para alcanzar mayoría, el grupo elige un nuevo líder y continúa operando; en caso contrario, el rango deja de aceptar escrituras hasta que se restablezca el quórum [7].

3.4. Modelo de programación paralela y leyes de escalamiento

El modelo RDD (*Resilient Distributed Dataset*) de Apache Spark abstrae una colección distribuida e inmutable de datos, con linaje de transformaciones, evaluación perezosa (*lazy evaluation*) y tolerancia a fallos mediante recómputo del linaje en lugar de replicación física [8]. Sobre este modelo se construye la API de DataFrames utilizada en el pipeline de la Sección 6.

La Ley de Amdahl expresa el *speedup* teórico máximo alcanzable al paralelizar un programa con una fracción p paralelizable sobre N procesadores, dejando $(1 - p)$ como fracción estrictamente secuencial:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}} \quad (2)$$

La ley predice un límite asintótico de $S(N) \rightarrow 1/(1 - p)$ cuando $N \rightarrow \infty$, lo que implica que, para fracciones secuenciales incluso pequeñas, el beneficio marginal de añadir más núcleos decae rápidamente [12]. Gustafson replantea esta lectura observando que, en la práctica, el volumen de trabajo suele escalar junto con los recursos disponibles en lugar de permanecer fijo, lo que produce una estimación de *speedup* escalado distinta y menos pesimista para cargas de trabajo que crecen con el tamaño del clúster [13]. Como se explica en la Sección 7, este proyecto no cuenta todavía con mediciones ejecutadas de N y $t(N)$ que permitan ajustar p empíricamente; la Ecuación (2) se documenta aquí como el marco que se aplicará cuando el protocolo experimental se ejecute.

Para ilustrar la lectura de la Ecuación (2) con valores hipotéticos –no medidos– considérese un pipeline con $p = 0.8$ de fracción paralelizable, un valor plausible para una carga dominada por combinaciones (*joins*) y agregaciones sobre particiones independientes. El límite asintótico sería $S(\infty) = 1/(1 - 0.8) = 5$, y ya con $N = 8$ núcleos se alcanzaría $S(8) = 1/(0.2 + 0.8/8) = 3.33$, es decir, el 66.7% del *speedup* máximo teórico. La pregunta obligatoria de la defensa (Anexo B, ítem 3 de la guía de la entrega) –con cuántos núcleos se alcanza el 90% del *speedup* máximo– se resuelve despejando N de $S(N) = 0.9 \cdot S(\infty)$ en la Ecuación (2), lo que para $p = 0.8$ arroja $N \approx 36$ núcleos; valores de p más altos reducen drásticamente ese requisito. Este cálculo se reproducirá con el p efectivamente estimado a partir de las mediciones del protocolo (Sección 7) una vez ejecutado.

Formalmente, Gustafson define el *speedup escalado* $S_g(N)$ asumiendo que el tiempo secuencial se mantiene fijo mientras el tamaño del problema crece proporcionalmente a N :

$$S_g(N) = N - \alpha \cdot (N - 1) \quad (3)$$

donde α es la fracción de tiempo que permanece secuencial en la ejecución paralela (no en la secuencial, a diferencia de p en Amdahl). La diferencia conceptual entre ambas leyes es relevante para este proyecto porque el pipeline de reservas procesa un volumen de datos que, en producción, crecería con el uso real del sistema y no permanecería fijo; la Sección 8

retoma esta distinción al interpretar qué ley es la lectura más apropiada para el dominio de SCLI.

4. Diseño del esquema distribuido

El microservicio `reservas-solicitudes-service` es propietario de cuatro tablas del dominio de reservas: `solicitudes_reserva`, `reservas`, `historial_solicitudes` y `bloqueos_agenda`, más una quinta tabla de configuración (`configuraciones_reserva`) que fija parámetros globales de anticipación mínima y máxima, duración permitida y política de fines de semana. El esquema vigente, heredado de la entrega anterior, se crea mediante migraciones Flyway y conserva claves primarias UUID simples; los identificadores de usuarios, docentes, laboratorios, materias y periodos lectivos son referencias externas por UUID, sin claves foráneas entre los cuatro microservicios del sistema, preservando así el patrón de base de datos por microservicio establecido en E2.

4.1. Modelo de datos y patrón de acceso

Las relaciones persistentes dentro del límite del microservicio son: una solicitud puede producir una reserva y cada reserva pertenece a una única solicitud (relación 1:1 diferida en el tiempo, ya que una solicitud puede existir sin haber sido aprobada todavía); una solicitud tiene múltiples eventos de historial (relación 1:N con `historial_solicitudes`); y un bloqueo de agenda no tiene clave foránea hacia las otras tablas, pero participa en las mismas consultas operativas mediante laboratorio, fecha y franja horaria. El [Listado 1](#) reproduce el fragmento central del esquema ejecutable aplicado en el clúster, correspondiente a la tabla `solicitudes_reserva`.

```
1 CREATE TABLE solicitudes_reserva (
2   id UUID DEFAULT gen_random_uuid(),
3   solicitante_id UUID NOT NULL,
4   docente_id UUID NOT NULL,
5   laboratorio_id UUID NOT NULL,
6   materia_id UUID NOT NULL,
7   periodo_lectivo_id UUID NOT NULL,
8   fecha_reserva DATE NOT NULL,
9   hora_inicio TIME NOT NULL,
10  hora_fin TIME NOT NULL,
11  numero_participantes INTEGER NOT NULL,
12  estado VARCHAR(30) NOT NULL
13   DEFAULT 'PENDIENTE',
14  creada_en TIMESTAMPTZ NOT NULL
15   DEFAULT CURRENT_TIMESTAMP,
16  version BIGINT NOT NULL DEFAULT 0,
17  CONSTRAINT pk_solicitudes_reserva
18   PRIMARY KEY (id),
19  CONSTRAINT ck_solicitudes_reserva_horas
20   CHECK (hora_fin > hora_inicio),
21  CONSTRAINT ck_solicitudes_reserva_estado
22   CHECK (estado IN ('PENDIENTE',
23                     'EN_REVISION', 'APROBADA',
24                     'RECHAZADA', 'CANCELADA', 'EXPIRADA'))
25 );
```

Listing 1: Fragmento de `db/schema.sql`: definición de `solicitudes_reserva`.

Nótese la columna `version`, empleada para control de concurrencia optimista a nivel de aplicación, y la restric-

Cuadro 1: Claves de partición temporal por tabla (ADR-003).

Tabla	Columna temporal
<code>solicitudes_reserva</code>	<code>fecha_reserva</code>
<code>reservas</code>	<code>fecha_reserva</code>
<code>historial_solicitudes</code>	<code>fecha_hora</code>
<code>bloqueos_agenda</code>	<code>fecha</code>

ción CHECK sobre el estado, que documenta en el propio esquema el ciclo de vida de una solicitud sin depender exclusivamente de la capa de aplicación para mantener la integridad de ese invariante. Las consultas de disponibilidad, calendario y detección de conflictos acotan los datos por `laboratorio_id` y por rangos temporales; el historial se consulta por `solicitud_id` y se ordena por `fecha_hora`. Este patrón de acceso es el que motiva directamente la estrategia de fragmentación descrita a continuación.

4.2. Estrategia de fragmentación (ADR-003)

Las consultas operativas dominantes del dominio – disponibilidad de laboratorios, calendario y detección de conflictos – acotan los datos por `laboratorio_id` y por rangos de fecha. Se evaluaron cinco alternativas de fragmentación antes de decidir:

- **Distribución automática por UUID:** sin estrategia explícita de fragmentación temporal ni base para administrar retención por periodo.
- **Hash del identificador:** distribuye uniformemente las escrituras pero no mejora la poda de consultas por fecha y rompe la alineación natural entre agenda, solicitudes y reservas de un mismo periodo.
- **Por laboratorio:** se aproxima a las consultas de disponibilidad, pero la actividad de los laboratorios no es uniforme y concentraría carga en los más usados.
- **Por estado:** los estados cambian durante el ciclo de vida de una solicitud, lo que movería filas entre particiones sin beneficiar las consultas históricas.
- **Por rangos temporales (adoptada):** coincide con los filtros existentes y permite administrar periodos cerrados.

Se adoptó `PARTITION BY RANGE` sobre índices temporales, con intervalos trimestrales de calendario semiabierto (inicio incluido, fin excluido), sin sustituir las claves primarias UUID existentes. La [Tabla 1](#) resume la columna de partición asignada a cada tabla.

La fecha de creación (`solicitudes_reserva.creada_en`) se descartó como clave de fragmentación porque las operaciones críticas consultan la fecha *reservada*, no la fecha de creación; se conserva como dato de auditoría. Los nombres lógicos y los límites trimestrales de `solicitudes_reserva`, `reservas` y `bloqueos_agenda` se alinean para que los datos de una

misma fecha de agenda pertenezcan al mismo intervalo lógico. `historial_solicitudes` conserva su propia partición por `fecha_hora`, sin duplicar `fecha_reserva` solo para forzar colocación, evitando así una redundancia que introduciría una nueva regla de consistencia a mantener.

4.2.1. Particiones de cobertura y diseño de índices

El ADR-003 exige que cada ciclo de implementación mantenga, además de las particiones trimestrales operativas, una partición anterior de cobertura y una partición futura de cobertura, de modo que fechas fuera del calendario previsto no generen errores de inserción en lugar de degradar silenciosamente el rendimiento de poda de rangos. Los índices temporales deberán anteponer la columna de partición y continuar con las columnas efectivamente usadas por las consultas: fecha, laboratorio y horario para solicitudes, reservas y bloqueos; fecha del evento y solicitud para el historial. El ADR-003 establece explícitamente que, antes de aplicar esta estrategia, el equipo debe verificar con `EXPLAIN` que las consultas de disponibilidad y calendario efectivamente usen los índices particionados y que exista poda de particiones (*partition pruning*); esa verificación –al depender del DDL todavía no materializado– es también trabajo pendiente hacia la E4.

4.2.2. Dos niveles de colocación

El ADR-003 distingue explícitamente dos niveles de colocación que es importante no confundir. El primer nivel es la *alineación lógica*: que los nombres y límites trimestrales de las particiones de `solicitudes_reserva`, `reservas` y `bloqueos_agenda` coincidan, de modo que una consulta de agenda para una fecha determinada recaiga conceptualmente en el mismo intervalo en las tres tablas. El segundo nivel es la *colocación física*: que CockroachDB efectivamente almacene esos rangos en los mismos nodos o en nodos cercanos, lo cual requiere políticas de localidad o de zona adicionales que el ADR-003 explícitamente deja fuera de su alcance y remite a una decisión posterior sustentada en la topología definitiva del clúster. Confundir ambos niveles sería un error conceptual grave: la alineación lógica no garantiza por sí sola ninguna mejora de latencia si las particiones correspondientes terminan distribuidas en nodos distintos por el asignador automático de CockroachDB.

Es importante precisar el alcance real de esta decisión a la fecha de este informe: el ADR-003 fue redactado y aprobado internamente por el equipo, pero el DDL correspondiente (las cláusulas `PARTITION BY RANGE` concretas, con sus límites de calendario y particiones de cobertura) todavía no se materializó en `db/schema.sql`. El esquema efectivamente aplicado en el clúster (Sección 5) crea las cinco tablas del dominio sin particiones activas; la fragmentación queda documentada como decisión arquitectónica lista para su siguiente subpaso de implementación, tal como exige el principio de complementariedad E3 → E4 enunciado en la introducción.

4.3. Estrategia de replicación (ADR-004)

El ADR-004 completa la decisión de colocación física dejada abierta por el ADR-003, fijando la política de réplicas para el clúster CockroachDB E3 mediante el comando de la Lista 2.

```
1 ALTER RANGE default CONFIGURE ZONE
2 USING num_replicas = 3;
```

Listing 2: Política de replicación del clúster E3 (ADR-004).

Se descartaron una y dos réplicas por no ofrecer mayoría capaz de sobrevivir a la caída de un nodo (con dos réplicas, la mayoría necesaria es de dos, de modo que la pérdida de cualquiera de las dos impide confirmar escrituras), y se descartaron cinco o más réplicas por exceder la topología de tres nodos disponible en esta entrega sin aportar diversidad efectiva de colocación. Con $f = 3$, el quórum de escritura es 2, que es exactamente el comportamiento verificado en la Sección 5.7.

4.3.1. Alcance de la política y su interpretación correcta

El ADR-004 advierte explícitamente contra una interpretación incorrecta de su propio alcance: el comando `ALTER RANGE default CONFIGURE ZONE` configura una política a nivel de *clúster*, no una propiedad de una tabla o de una aplicación en particular. Para que la política de tres réplicas aplique efectivamente solo a los datos de `reservas-solicitudes-service` –y no arrastre, sin quererlo, otros datos que eventualmente se aloje en el mismo clúster– el ADR-004 establece como condición que el clúster E3 permanezca dedicado exclusivamente a ese microservicio mientras el alcance actual esté vigente. Si en una evolución futura el clúster llegara a alojar datos de otros dominios, el ADR exige reevaluar el efecto global de la política `RANGE default` o sustituirla por políticas de zona con un alcance más específico por tabla o por base de datos. Esta salvedad es relevante para la Entrega 4: cualquier decisión de escalar el clúster a otros microservicios deberá revisar primero esta condición.

4.3.2. Relación entre replicación y consenso

El ADR-004 explica la mecánica interna que conecta la política de replicación con el algoritmo de consenso descrito en la Sección 3: cada rango de CockroachDB constituye un grupo de replicación independiente cuyas réplicas participan en una instancia de Raft. Una de las réplicas mantiene el liderazgo del grupo y coordina las propuestas de cambio; el líder envía *heartbeats* periódicos a los seguidores para mantener sus registros replicados, y si esos *heartbeats* dejan de llegar, los seguidores se convierten en candidatos tras un tiempo de espera aleatorio y celebran una nueva elección. Con tres réplicas, el grupo tolera la pérdida de una sin dejar de poder elegir líder y confirmar escrituras; con dos réplicas activas supervivientes tras la caída de la tercera, el grupo sigue teniendo mayoría (2 de 3) y continúa operando con normalidad, que es exactamente el comportamiento capturado en la Figura 9.

El clúster E3 se reserva, en el alcance vigente, exclusivamente para los datos de reservas-solicitudes-service; los servicios de autenticación, usuarios y académico-laboratorios continúan sobre instancias CockroachDB de un solo nodo cada uno, sin cambios respecto de la entrega anterior.

5. Clúster CockroachDB y su verificación

5.1. Arquitectura general del sistema

El sistema SCLI completo, tras esta entrega, se compone de cuatro microservicios Spring Boot (auth-service, usuarios-service, academico-laboratorios-service y reservas-solicitudes-service), un api-gateway que enruta las peticiones del frontend, y un frontend en React (scli-front). Los tres primeros microservicios conservan cada uno su propia instancia CockroachDB de un solo nodo, sin cambios en esta entrega; el cuarto es el único que consume el clúster distribuido de tres nodos que se describe en esta sección. Esta decisión de alcance –no migrar los cuatro microservicios al clúster distribuido en el mismo ciclo– se tomó deliberadamente para acotar el riesgo de la entrega a un único punto de integración crítico, dejando la migración de los demás servicios como trabajo candidato para ciclos posteriores si el patrón de acceso de esos dominios llegara a justificarlo.

La Figura 1 resume esta topología. Es una representación de despliegue equivalente, a nivel C4, al diagrama de nivel 3 (componentes y contenedores) que el equipo mantiene en docs/evidencias/diagramas_arquitectura/arquitecturaScli.drawio; se reproduce aquí en TeX nativo para que el documento compile sin dependencias externas de imágenes vectoriales.

5.2. Despliegue con Docker Compose

El clúster E3 se añadió al docker-compose.yml heredado de la entrega anterior como tres servicios independientes (crdb-e3-1, crdb-e3-2, crdb-e3-3), cada uno con volumen de datos propio y descubrimiento mutuo mediante la bandera –join, más un cuarto servicio de inicialización (crdb-e3-init) responsable de ejecutar cockroach init y aplicar el esquema una única vez. Los tres nodos comparten la red interna scli-network del proyecto. El Listado 3 muestra la definición de uno de los tres nodos y del servicio de inicialización, tal como está versionado en el repositorio.

```
1 crdb-e3-1:
2   image: cockroachdb/cockroach:v26.2.0
3   command:
4     - start
5     - --insecure
6     - --join=crdb-e3-1,crdb-e3-2,
7       crdb-e3-3
8   ports:
9     - "${CRDB_E3_1_SQL_PORT}:26257"
10    - "${CRDB_E3_1_HTTP_PORT}:8080"
11   volumes:
12     - crdb-e3-1-data:/cockroach/
```

```
13   cockroach-data
14   networks: [scli-network]
15
16 crdb-e3-init:
17   depends_on: [crdb-e3-1, crdb-e3-2,
18     crdb-e3-3]
19   entrypoint: /bin/sh
20   # cockroach init + carga condicional
21   # de db/schema.sql si no existe
22   volumes:
23     - ./db/schema.sql:/schema.sql:ro
24   networks: [scli-network]
```

Listing 3: Definición de un nodo del clúster E3 y de su servicio de inicialización (docker-compose.yml).

El servicio crdb-e3-init verifica, antes de aplicar el esquema, si la tabla configuraciones_reserva ya existe en reservas_db; en ese caso omite la carga para que reinicios sucesivos del entorno no fallen por intentar recrear un esquema ya presente. Esta guarda de idempotencia es la misma razón por la que, como se documentó en la Sección 4, la fragmentación temporal del ADR-003 no puede materializarse simplemente añadiendo cláusulas PARTITION BY RANGE al final del archivo: requiere una migración explícita que distinga el esquema ya aplicado del esquema fragmentado, lo que se documenta como trabajo pendiente hacia la E4. La Figura 2 muestra los tres contenedores del clúster E3 activos junto al resto de servicios del sistema.

5.3. Integración con aislamiento serializable

El Módulo A de la guía de la entrega exige explicitar transacciones con nivel SERIALIZABLE al refactorizar el microservicio principal. En reservas-solicitudes-service, las operaciones que modifican el estado de una solicitud o de una reserva –aprobar, rechazar, cancelar, crear una reserva a partir de una solicitud aprobada– están anotadas explícitamente con @Transactional(isolation = Isolation.SERIALIZABLE) en la capa de servicio, como muestra el Listado 4.

```
1 @Override
2 @Transactional(
3     isolation = Isolation.SERIALIZABLE)
4 public ReservaResponse aprobar(
5     UUID id,
6     AprobarSolicitudRequest request,
7     String claveIdempotencia,
8     UUID usuarioAutenticadoId) {
9     SolicitudReserva solicitud =
10     solicitudReservaRepository
11     .findByIdForUpdate(id)
12     .orElseThrow(() ->
13         new IllegalArgumentException(
14             "No existe la solicitud"));
15     // ... validación de conflicto de
16     // horario y transición de estado
17 }
```

Listing 4: Transacción serializable en la aprobación de una solicitud (SolicitudReservaServiceImpl.java).

Este nivel de aislamiento es intencional y directamente relacionado con el teorema CAP discutido en la Sección 3: CockroachDB implementa SERIALIZABLE como su único nivel de aislamiento transaccional –a diferencia de PostgreSQL, que ofrece varios niveles y usa READ COMMITTED

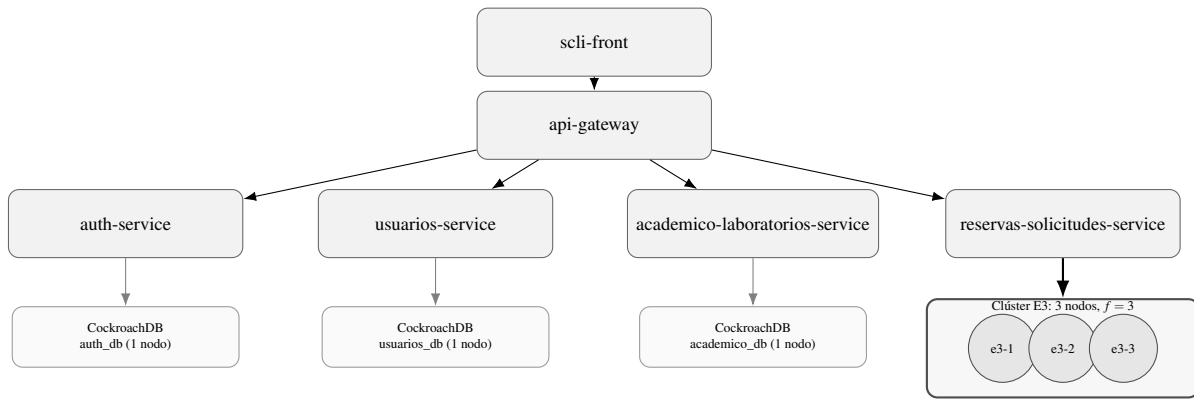


Figura 1: Arquitectura de despliegue del sistema SCLI tras la Entrega 3. Tres microservicios conservan su base de datos mono-nodo heredada de E2; el microservicio de reservas y solicitudes es el único que consume el clúster CockroachDB distribuido de tres nodos añadido en esta entrega.

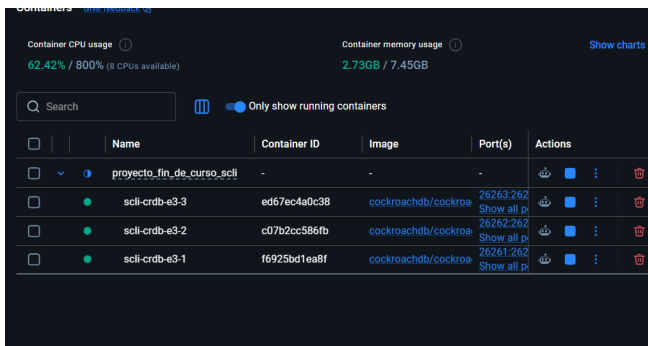


Figura 2: Los tres nodos `scli-crdb-e3-*` activos en Docker, junto al resto de servicios del sistema SCLI.

por defecto— precisamente para sostener la consistencia fuerte que su posicionamiento como sistema CP promete. La combinación de `findByIdForUpdate` (que adquiere un bloqueo de fila) con aislamiento serializable es la salvaguarda de aplicación contra dos aprobaciones concurrentes de la misma solicitud, un escenario de concurrencia real del dominio de reservas de laboratorios que no se limita a una preocupación teórica: dos docentes podrían intentar aprobar la misma solicitud casi simultáneamente desde sesiones distintas del sistema.

5.4. Reproducibilidad y trazabilidad

Todo el trabajo de esta entrega se desarrolló en la rama `feature/entrega-3`, integrada a `main` mediante solicitudes de extracción (*pull requests*) individuales por integrante—con ramas propias `feature/entrega-3-farinango`, `feature/entrega-3-villamarin`, `feature/entrega-3-urbina` y `feature/entrega-3-vinueza`— y revisión cruzada antes de cada fusión, siguiendo el mismo flujo de trabajo establecido en la entrega anterior. El archivo `.env.example` del repositorio documenta las variables de entorno necesarias para levantar el sistema completo, incluidas las credenciales del clúster E3, y el comando `docker compose up`

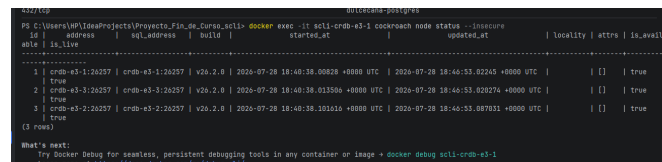


Figura 3: Estado de los tres nodos del clúster E3 (`cockroach node status`): los tres reportan `is_live=true`.

—d desde la raíz del repositorio construye y levanta los cuatro microservicios, el `api-gateway`, las tres instancias CockroachDB de un solo nodo y el clúster de tres nodos en un único paso, sin configuración manual adicional. La carga de datos de referencia (Sección 5.6) es determinista: el generador de las 100 000 filas de `solicitudes_reserva` produce siempre el mismo conjunto de datos a partir de una semilla fija, lo que permite repetir la verificación de tolerancia a fallos sobre exactamente el mismo volumen de datos en ejecuciones futuras.

5.5. Verificación de estado y del esquema aplicado

El comando `cockroach node status -insecure`, ejecutado sobre el nodo `crdb-e3-1`, confirma que los tres nodos se reconocen mutuamente y reportan `is_live=true` e `is_available=true` (Figura 3). El esquema aplicado en la base de datos `reservas_db` contiene las cinco tablas del dominio de reservas más la tabla de control de migraciones de Flyway (Figura 4), confirmando que la inicialización del clúster se completó correctamente.

La consulta `SHOW ZONE CONFIGURATION FOR RANGE default` confirma que la política de replicación del ADR-004 quedó efectivamente configurada con `num_replicas = 3` (Figura 5).

5.6. Carga de datos de referencia

Se cargó la tabla `solicitudes_reserva` con 100 000 filas mediante un generador reproducible, en lotes de 10 000

Figura 4: Tablas del esquema `reservas_db` aplicado en el clúster E3.

Figura 5: Configuración de zona del rango default, con `num_replicas = 3` aplicado.

filas. La Figura 6 muestra la verificación del conteo total mediante `SELECT COUNT(*)` tras completar la carga.

5.6.1. Metodología de generación

El repositorio versiona diez archivos `db/seeds_01.sql` a `db/seeds_10.sql` de 10 000 sentencias `INSERT` cada uno, más un `db/seeds.sql` de referencia con los valores de configuración mínimos necesarios para que las restricciones `CHECK` del esquema –como la verificación `hora_fin > hora_inicio`, mostrada en el Listado 1– se satisfagan en cada fila generada. Dividir la carga en diez archivos de 10 000 filas, en lugar de un único archivo de 100 000, respondió a una razón práctica: permite verificar el conteo parcial tras cada lote (como se documentó en la bitácora del equipo con la captura de la carga del primer lote) antes de continuar con el siguiente, detectando así cualquier violación de restricción temprano en el proceso en lugar de al final de una transacción de gran tamaño. Los identificadores `UUID` de solicitante, docente, laboratorio, materia y periodo lectivo referenciados en las filas generadas son valores fijos y deterministas, no extraídos de los otros tres microservicios del sistema; esto es coherente con el diseño sin claves foráneas entre bases de datos descrito en la Sección 4, pero implica que el conjunto de datos de referencia es sintético y no representa necesariamente la distribución real de solicitudes que produciría el sistema en operación, limitación que se retoma en la Sección 8.

5.7. Verificación experimental de tolerancia a fallos

Siguiendo el protocolo del Módulo D de la guía de la entrega, se ejecutó la siguiente secuencia sobre el clúster con las 100 000 filas ya cargadas: (i) consulta `SELECT COUNT(*)`

Figura 6: Verificación del conteo total (100 000 filas) en `solicitudes_reserva` tras la carga por lotes.

Figura 7: Fragmento de `db/seeds_05.sql`: sentencia `INSERT` de configuración de reservas con los parámetros mínimos y máximos exigidos por las restricciones del esquema.

con los tres nodos operativos; (ii) detención abrupta del nodo `scli-crdb-e3-2` con `docker stop`; (iii) reconsulta con un nodo caído; (iv) reincorporación del nodo detenido y reconsulta tras la reconvergencia; (v) repetición del experimento deteniendo dos de los tres nodos. La Tabla 2 resume los resultados de latencia observados.

Las Figuras 9 y 10 muestran la evidencia de consola de ambos escenarios de fallo. Con un nodo caído, `cockroach node status` reporta al nodo detenido con `is_live=false` mientras los otros dos permanecen disponibles, y la consulta se sirve con éxito porque el grupo Raft de cada rango conserva su mayoría de dos réplicas de tres. Con dos nodos caídos, ningún rango conserva mayoría y la consulta no puede completarse hasta restablecer el quórum, consistente con la Sección 3.

Tras reiniciar el nodo detenido, el clúster recuperó su estado de los tres nodos operativos (Figura 11), confirmando que la reconvergencia de Raft no requirió intervención manual adicional más allá de reiniciar el proceso del nodo.

5.7.1. Evidencia de las mediciones de latencia

Las Figuras 12–14 muestran las capturas de consola de PowerShell (`Measure-Command`) que respaldan cada una de las tres latencias numéricas de la Tabla 2, medidas con el mismo comando `SELECT COUNT(*)` sobre `solicitudes_reserva` en los tres escenarios de disponibilidad del clúster.

5.7.2. Nota metodológica sobre la latencia reportada

Las cuatro latencias de la Tabla 2 corresponden a una medición puntual por escenario ($n = 1$), no a un promedio de repeticiones. El protocolo experimental del Módulo F (Sección 7)

```
PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker exec -it scli-crdb-e3-1 cockroach sql --insecure --database=reservas_db -e "SELECT COUNT(*) FROM solicitudes_reserva;"
Count
-----
10000
(1 row)

Time: 22ms

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug scli-crdb-e3-1
Learn more at https://docs.docker.com/go/debug-cli/
```

Figura 8: Verificación del conteo parcial (10 000 filas) tras cargar el primer lote de db/seeds_01.sql.

Cuadro 2: Latencia de SELECT COUNT(*) por escenario de disponibilidad del clúster.

Escenario	Resultado	Latencia
Clúster operativo (3 nodos)	Consulta exitosa	2604.54 ms
Un nodo fuera de servicio	Consulta exitosa	2681.40 ms
Clúster recuperado (3 nodos)	Consulta exitosa	2906.34 ms
Dos nodos fuera de servicio	Sin quórum	No aplica

exige $r = 10$ repeticiones por nivel con intervalo de confianza al 95 %; esa ejecución sistemática todavía no se realizó para este escenario de tolerancia a fallos, y se documenta explícitamente como trabajo pendiente en la Sección 8 en lugar de presentar una única observación como si fuera una serie estadística.

5.8. Pruebas de integración con Testcontainers

El Módulo A de la guía de la entrega exige actualizar las pruebas de integración para levantar el clúster con *Testcontainers* en lugar de depender de una base de datos local. `usuarios-service` y `reservas-solicitudes-service` incorporan sendas pruebas de integración (`CockroachFlywayIntegrationTest` y `CockroachFlywayIntegrationTests`, respectivamente) que instancian un contenedor `CockroachContainer` de la biblioteca `testcontainers-java` en tiempo de ejecución de la prueba, registran su URL JDBC dinámicamente mediante `@DynamicPropertySource`, habilitan las migraciones de Flyway sobre ese contenedor efímero, y verifican que el esquema se aplique correctamente contando las filas de una tabla de referencia poblada por las migraciones (once perfiles, en el caso de `usuarios-service`). Esta estrategia—contenedor CockroachDB efímero por ejecución de prueba, en lugar de una base compartida entre ejecuciones—es la que garantiza que las pruebas de integración no dependan de un estado externo persistente y puedan ejecutarse de forma aislada en el flujo de integración continua heredado de la entrega anterior. Nótese que estas pruebas verifican la aplicación correcta del esquema *mono-nodo* de cada microservicio; la extensión de este mismo patrón para levantar el clúster de tres nodos del ADR-004 dentro de una prueba de integración—en lugar de contra el entorno Docker Compose persistente usado en la Sección 5.7—queda como mejora concreta para el ciclo de la E4.

```
PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker exec -it scli-crdb-e3-1 cockroach sql --insecure --database=reservas_db -e "SELECT COUNT(*) FROM solicitudes_reserva;"
Count
-----
10000
(1 row)

Time: 22ms

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug scli-crdb-e3-1
Learn more at https://docs.docker.com/go/debug-cli/

PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker stop scli-crdb-e3-2
scli-crdb-e3-2

PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker exec -it scli-crdb-e3-1 cockroach node status --insecure
id | address | sql_address | build | started_at | updated_at | locality | attr | is_a
----|-----|-----|-----|-----|-----|-----|-----|
1 | crdb-e3-1:26257 | crdb-e3-1:26257 | v20.2.0 | 2020-07-28 19:48:31.225480 +0000 UTC | 2020-07-28 22:41:57.475039 +0000 UTC | {} | true
2 | crdb-e3-2:26257 | crdb-e3-2:26257 | v20.2.0 | 2020-07-28 19:48:31.128758 +0000 UTC | 2020-07-28 22:41:57.562211 +0000 UTC | {} | true
3 | crdb-e3-2:26257 | crdb-e3-2:26257 | v20.2.0 | 2020-07-28 19:48:31.051986 +0000 UTC | 2020-07-28 22:41:57.562211 +0000 UTC | {} | false
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug scli-crdb-e3-1
Learn more at https://docs.docker.com/go/debug-cli/
```

Figura 9: Un nodo detenido (`crdb-e3-2`): el clúster conserva quórum y la consulta se ejecuta con éxito.

```
PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker stop scli-crdb-e3-1
scli-crdb-e3-1

PS C:\Users\jrodriguez\Projects_Fin_de_Curso_sqli> docker exec -it scli-crdb-e3-1 cockroach node status --insecure
id | address | sql_address | build | started_at | updated_at | locality | attr | is_a
----|-----|-----|-----|-----|-----|-----|-----|
1 | crdb-e3-1:26257 | crdb-e3-1:26257 | v20.2.0 | 2020-07-28 19:48:31.225480 +0000 UTC | 2020-07-28 22:47:21.658539 +0000 UTC | {} | false
2 | crdb-e3-2:26257 | crdb-e3-2:26257 | v20.2.0 | 2020-07-28 19:48:31.128758 +0000 UTC | 2020-07-28 22:47:21.565202 +0000 UTC | {} | false
3 | crdb-e3-2:26257 | crdb-e3-2:26257 | v20.2.0 | 2020-07-28 19:48:31.051986 +0000 UTC | 2020-07-28 22:47:21.424633 +0000 UTC | {} | false
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug scli-crdb-e3-1
Learn more at https://docs.docker.com/go/debug-cli/
```

Figura 10: Dos nodos detenidos: los tres nodos reportan `is_live=false` y el clúster pierde disponibilidad de escritura.

6. Pipeline analítico paralelo

6.1. Arquitectura del pipeline

El pipeline de PySpark (`spark/pipeline.py`) obtiene sus fuentes directamente del clúster CockroachDB E3 mediante el conector JDBC de PostgreSQL—protocolo con el que CockroachDB es compatible—y aplica exactamente cinco transformaciones sobre el dominio de reservas, en el mismo orden en su línea base equivalente escrita en pandas (`spark/baseline.py`). El Listado 5 muestra la composición de las transformaciones en la versión Spark.

```
1 def construir_pipeline(fuentes):
2     reservas_activas = filtrar_reservas_activas(
3         fuentes["reservas"])
4     datos = unir_solicitudes_con_reservas(
5         fuentes["solicitudes_reserva"],
6         reservas_activas)
7     datos = agregar_dimensiones_temporales(datos)
8     datos = agregar_participantes_por_periodo(
9         datos)
10    return categorizar_numero_participantes(
11        datos)
```

Listing 5: Composición de las cinco transformaciones del pipeline (`spark/pipeline.py`).

Las cinco transformaciones, en orden de aplicación, son:

1. **Filtrado:** conserva únicamente las reservas en estado PROGRAMADA o EN_CURSO, descartando canceladas, rechazadas o expiradas.
2. **Combinación (join):** relaciona cada reserva activa con los datos de su solicitud de origen mediante una unión interna sobre `solicitud_id`, seleccionando y renombrando las columnas relevantes de ambas tablas.

```
PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_sqli>
PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_sqli> docker exec -it sqli-crdb-e3-1 cockroach node status --insecure
node | sqli-crdb-e3-1
-----|-----
id | address | sql_address | build | started_at | updated_at | locality | attr | is_available
-----|-----
1 | crdb-e3-1:26257 | crdb-e3-1:26257 | v20.2.0 | 2020-07-28 19:48:31.225486 +0000 UTC | 2020-07-28 22:49:52.998318 +0000 UTC | [] | true
2 | crdb-e3-1:26257 | crdb-e3-1:26257 | v20.2.0 | 2020-07-28 22:49:19.264672 +0000 UTC | 2020-07-28 22:49:49.010476 +0000 UTC | [] | true
3 | crdb-e3-2:26257 | crdb-e3-2:26257 | v20.2.0 | 2020-07-28 22:49:08.432678 +0000 UTC | 2020-07-28 22:49:48.496492 +0000 UTC | [] | true
(3 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image • docker debug sqli-crdb-e3-1
Learn more at https://docs.docker.com/go/psdev-sli/

PS C:\Users\HP\IdeaProjects\Proyecto_Fin_de_Curso_sqli>
```

Figura 11: Recuperación completa del clúster tras reiniciar el nodo detenido.

```
PS C:\Users\USER\Desktop\Proyecto_Fin_de_Curso_sqli> Measure-Command {
>> docker exec -it sqli-crdb-e3-1 cockroach sql --insecure --database=reservas_db -e "SELECT COUNT(*) FROM solicitudes_reserva;"
>> }

Days : 0
Hours : 0
Minutes : 0
Seconds : 2
Milliseconds : 481
Ticks : 26813962
TotalDays : 0.30367824074076-05
TotalHours : 0.0074483227777778
TotalMinutes : 0.0446899366666667
TotalSeconds : 2.6813962
TotalMilliseconds : 2681.3962
```

Figura 12: Latencia con los tres nodos operativos: 2604.54 ms.

- Transformación de tipos temporales:** normaliza fecha_reserva a tipo fecha y deriva año, trimestre y primer día del mes como columnas de calendario.
- Agregación con ventana:** calcula, mediante una función de ventana particionada por laboratorio, año y trimestre, la suma de participantes por periodo (participantes_laboratorio_trimestre).
- Transformación de aprendizaje automático:** aplica un Bucketizer de pyspark.ml.feature para discretizar el número de participantes de cada reserva en cuatro intervalos numéricos (< 10 , $[10, 20)$, $[20, 30)$, ≥ 30).

La línea base en pandas replica exactamente la misma secuencia lógica: la combinación se implementa con DataFrame.merge, la agregación de ventana con groupby(...).transform('sum') y la discretización con pandas.cut usando los mismos límites de intervalo que el Bucketizer de Spark, de modo que ambos tratamientos son funcionalmente equivalentes y comparables bajo el mismo conjunto de datos. Ambos scripts exportan su resultado en formato Parquet. El Listado 6 muestra la función equivalente de agregación por ventana en la línea base, que ilustra la correspondencia deliberada entre ambas implementaciones.

```
1 def agregar_participantes_por_periodo(
2     datos):
3     grupos = ["laboratorio_id",
4              "anio_reserva",
5              "trimestre_reserva"]
6     resultado = datos.copy()
7     resultado[
8         "participantes_laboratorio_"
9         "trimestre"] = (
10         resultado
11         .groupby(grupos, dropna=False)
12         ["numero_participantes"]
13         .transform("sum")
14     )
15     return resultado
```

Listing 6: Agregación por ventana equivalente en la línea base pandas (spark/baseline.py).

```
PS C:\Users\USER\Desktop\Proyecto_Fin_de_Curso_sqli> Measure-Command {
>> docker exec -it sqli-crdb-e3-1 cockroach sql --insecure --database=reservas_db -e "SELECT COUNT(*) FROM solicitudes_reserva;"
>> }

Days : 0
Hours : 0
Minutes : 0
Seconds : 2
Milliseconds : 481
Ticks : 26813962
TotalDays : 0.30367824074076-05
TotalHours : 0.0074483227777778
TotalMinutes : 0.0446899366666667
TotalSeconds : 2.6813962
TotalMilliseconds : 2681.3962
```

Figura 13: Latencia con un nodo caído: 2681.40 ms.

```
PS C:\Users\USER\Desktop\Proyecto_Fin_de_Curso_sqli> Measure-Command {
>> docker exec -it sqli-crdb-e3-1 cockroach sql --insecure --database=reservas_db -e "SELECT COUNT(*) FROM solicitudes_reserva;"
>> }

Days : 0
Hours : 0
Minutes : 0
Seconds : 2
Milliseconds : 906
Ticks : 29063420
TotalDays : 0.34382193229206-05
TotalHours : 0.0080731722222222
TotalMinutes : 0.048439333333333
TotalSeconds : 2.906342
TotalMilliseconds : 2906.342
```

Figura 14: Latencia tras la recuperación completa: 2906.34 ms.

6.2. Configuración de lectura distribuida

La lectura JDBC del pipeline Spark se parametriza para leer en paralelo por particiones de la columna id, evitando que la extracción completa del clúster se sirva desde un único nodo lector. La configuración por defecto establece 16 particiones de lectura sobre el rango de identificadores del conjunto de datos, y la sesión de Spark se construye con spark.sql.shuffle.partitions ajustado explícitamente en lugar de usar el valor por defecto de 200 particiones, que resultaría sobredimensionado para un volumen de 10^5 filas y produciría una sobrecarga de coordinación desproporcionada frente al trabajo útil por partición. La cadena de conexión y las credenciales se externalizan mediante las variables de entorno RESERVAS_JDBC_URL, RESERVAS_DB_USERNAME y RESERVAS_DB_PASSWORD, con valores por defecto que apuntan a los tres puertos SQL locales del clúster E3 (26261, 26262, 26263), de modo que el controlador JDBC de PostgreSQL pueda alternar entre nodos si alguno no responde.

6.3. Detalle de la transformación de combinación

La segunda transformación del pipeline –la combinación entre solicitudes y reservas activas– ilustra una decisión de diseño relevante para la correspondencia entre ambas implementaciones: en lugar de unir todas las columnas de ambas tablas y arrastrar sufijos ambiguos, ambas versiones seleccionan y renombran explícitamente solo las catorce columnas necesarias para las transformaciones posteriores, tal como se aprecia en el Listado 7.

```
1 def unir_solicitudes_con_reservas(
2     solicitudes, reservas):
3     s = solicitudes.alias("solicitud")
4     r = reservas.alias("reserva")
5     return s.join(
6         r,
7         F.col("solicitud.id") ==
8         F.col("reserva.solicitud_id"),
9         "inner",
10    ).select(
11        F.col("reserva.id")
```

```

12     .alias("reserva_id"),
13     F.col("reserva.laboratorio_id"),
14     F.col("reserva.fecha_reserva"),
15     F.col("solicitud.solicitante_id"),
16     F.col("solicitud.docente_id"),
17     F.col("solicitud.numero_"
18         "participantes"),
19 )

```

Listing 7: Combinación explícita de columnas en `pipeline.py`.

Esta decisión –preferir una proyección explícita de columnas sobre un `join` sin seleccionar– reduce el volumen de datos que viaja entre etapas del plan de ejecución distribuido de Spark y elimina la ambigüedad de nombres de columna duplicados entre `solicitudes_reserva` y `reservas` (ambas tienen columnas `id`, `laboratorio_id` y `fecha_reserva`), que de otro modo obligaría a resolver sufijos automáticos tanto en la versión Spark como en la versión pandas de manera potencialmente inconsistente.

6.4. Estado de implementación

A la fecha de este informe, tanto `pipeline.py` como `baseline.py` están completos y son ejecutables de forma independiente contra el clúster E3, pero **ninguno de los dos se ha ejecutado todavía de manera sistemática** sobre el dataset de referencia de 100 000 filas cargado en la Sección 5. El Listado 8 resume el orquestador experimental (`experiments/run_comparison.py`) construido para automatizar esa comparación cuando se ejecute: alterna el orden de los dos tratamientos, descarta una iteración de calentamiento, muestrea CPU y memoria del árbol de procesos con `psutil`, y registra una observación por repetición conforme al esquema JSON `experiments/metrics/metric-schema.json`.

```

1  "treatments": [
2    {"id": "pandas-baseline",
3     "entrypoint": "spark/baseline.py"},
4    {"id": "pyspark-pipeline",
5     "entrypoint": "spark/pipeline.py"}
6  ],
7  "execution": {
8    "warmup_iterations": 1,
9    "measured_iterations": 5,
10   "alternate_treatment_order": true
11 }

```

Listing 8: Firma de los tratamientos configurados en `experiment-config.json`.

El archivo de resultados agregados (`experiments/metrics/results/comparacion.csv`) existe en el repositorio como una plantilla vacía –con sus 26 columnas declaradas y ninguna fila de datos– que el orquestador completará únicamente cuando se ejecuten repeticiones reales. No se presenta en este informe ninguna cifra de *speedup*, uso de CPU o memoria para el par pandas/PySpark, precisamente porque esa medición no existe todavía; sí se documenta el protocolo con el que se generará (Sección 7).

7. Protocolo experimental y resultados

7.1. Diseño del protocolo

El protocolo experimental (`experiments/protocolo-experimental.md`) sigue las recomendaciones de Jain [10] y de Wohlin *et al.* [9] para la comparación de sistemas de cómputo, y define, sin ejecutarlos todavía, los siguientes elementos:

Hipótesis. H_1 : el pipeline paralelo con N núcleos presenta un tiempo medio de ejecución menor que la línea base secuencial para $|D| \geq 5 \times 10^5$ registros; H_0 : no hay diferencia significativa.

Variable independiente. Motor de procesamiento (`pandas-baseline` o `pyspark-pipeline`).

Variables dependientes. Tiempo total de ejecución en milisegundos, utilización media y máxima de CPU, memoria residente media y máxima, filas procesadas y rendimiento en filas por segundo.

Variables controladas. Versión del conjunto de datos, estado de CockroachDB antes de cada tratamiento, equipo y recursos disponibles, versiones de Python, pandas, Java y Spark, configuración de Spark, número de repeticiones y calentamientos, y procesos externos activos durante la medición.

Diseño. Preparación de una instantánea identificable del conjunto de datos; registro del entorno y las versiones; iteraciones de calentamiento excluidas de los resultados; alternancia del orden de los tratamientos para reducir el sesgo de caché y de orden de ejecución; $r = 10$ repeticiones medidas por tratamiento (según el Anexo A de la guía de la entrega), de las cuales se descartan la primera y la última como calentamiento y enfriamiento; registro de una observación por ejecución conforme al esquema JSON, incluidas las ejecuciones fallidas.

Análisis planificado. Media $\bar{t}$, desviación típica s e intervalo de confianza al 95 % mediante $\bar{t} \pm 1.96 \cdot s / \sqrt{r}$; contraste con prueba t pareada si se cumple el supuesto de normalidad, o prueba de Wilcoxon en caso contrario.

7.2. Esquema de registro de observaciones

Cada ejecución medida se registra conforme al esquema JSON `experiments/metrics/metric-schema.json`, que declara como campos obligatorios un identificador único de ejecución (`run_id`), la versión del protocolo, el tratamiento (`pandas-baseline` o `pyspark-pipeline`), el identificador y la instantánea del conjunto de datos, marcas de tiempo de inicio y fin, duración en milisegundos, utilización de CPU media y máxima, memoria residente media y máxima, filas procesadas, rendimiento en filas por segundo, estado de la ejecución (`completed`, `failed` o `timeout`) y una descripción del entorno de ejecución (equipo, sistema operativo, modelo de CPU, número de núcleos lógicos, memoria del sistema y versiones de Python, pandas, Java y Spark). Esta granularidad permite, entre otras cosas, detectar si una diferencia de rendimiento observada se

debe al motor de procesamiento o a un cambio de entorno entre ejecuciones, y es la misma estructura que se reutilizará para las métricas Prometheus incrementales previstas para la E4.

Condiciones de validez de una repetición. Una repetición se considera válida cuando termina sin error y produce el resultado esperado; los dos tratamientos deben procesar la misma cantidad de filas de entrada para que la comparación sea funcionalmente equivalente; los datos fuente no deben modificarse entre tratamientos equivalentes; y una interrupción externa debe registrarse como ejecución fallida en lugar de eliminarse silenciosamente del conjunto de datos. El rendimiento se calcula siempre *a posteriori* a partir de filas procesadas y tiempo de ejecución medidos; el protocolo prohíbe explícitamente registrar valores estimados o interpolados en el archivo de resultados, precisamente para evitar la situación –que este informe busca no repetir– de presentar una cifra sin respaldo empírico directo.

7.3. Resultados disponibles

De los dos frentes experimentales de esta entrega, únicamente la verificación de tolerancia a fallos del clúster (Tabla 2, Sección 5.7) cuenta con evidencia medida, y se trata de una observación puntual por escenario, no de una serie con repeticiones. La comparación pandas/PySpark descrita en el protocolo anterior no tiene ejecuciones registradas: el archivo de resultados agregados está vacío y ningún valor de *speedup*, *p* estimado en la Ecuación (2), ni intervalo de confianza puede reportarse honestamente en este informe. Esta limitación se retoma como principal amenaza a la validez de conclusión en la Sección 8.

7.4. Estado de cumplimiento por módulo

La Tabla 3 resume, para los siete módulos técnicos definidos en la guía de la entrega (Módulos A a G), el estado alcanzado a la fecha de este informe, distinguiendo explícitamente entre lo completado, lo parcialmente completado y lo pendiente, en lugar de reportar un estado binario que ocultaría matices relevantes para la defensa oral.

8. Discusión

8.1. Interpretación

La verificación de tolerancia a fallos confirma, sobre datos y contenedores propios del equipo, el comportamiento teórico descrito en la Sección 3: con factor de replicación $f = 3$ y quórum de escritura 2, el clúster tolera exactamente la caída de un nodo y pierde disponibilidad de escritura al perder dos de tres réplicas por rango. El incremento moderado de latencia observado al caer un nodo (de 2604.54 ms a 2681.40 ms, un 2.9 % adicional) es consistente con el costo de que el grupo Raft opere con dos réplicas activas en lugar de tres, aunque una diferencia de este tamaño sobre una sola observación no permite descartar que sea ruido del entorno; una conclusión

Cuadro 3: Estado de cumplimiento de los módulos técnicos de la E3.

Módulo	Estado
A. Refactorización	Reservas migrado al clúster; los otros tres microservicios sin cambios (alcanza acotado, Sección 5).
B. Esquema distribuido	ADR-003 aprobado; DDL de partición pendiente.
C. Clúster Docker	Completo y verificado (Sección 5).
D. Tolerancia a fallos	Completo, con evidencia y bitácora; latencia puntual, no repetida.
E. Pipeline Spark	Código completo (5 transformaciones); sin ejecución sobre el dataset de referencia.
F. Protocolo experimental	Diseñado con detalle; sin repeticiones ejecutadas.
G. Métricas Prometheus	No iniciado en este ciclo.

estadísticamente sólida requeriría las repeticiones que el protocolo de la Sección 7 todavía no ejecutó. Un segundo dato observable en la Tabla 2 es que la latencia tras la recuperación del clúster (2906.34 ms) resultó mayor que la latencia con un nodo caído (2681.40 ms), en lugar de volver al valor previo al fallo (2604.54 ms). Esto es consistente con el proceso de reconvergencia de Raft: al reincorporarse, el nodo reiniciado debe recibir del líder los registros de operaciones que se acumularon durante su ausencia antes de poder servir lecturas consistentes, lo que añade sobrecarga temporal justo en la ventana de la medición; no se interpreta como un problema del clúster, sino como el costo esperado de la recuperación, y sería interesante –en una repetición futura del protocolo– medir cuánto tiempo tarda la latencia en volver a su línea base tras la reincorporación.

Respecto de la elección entre la Ley de Amdahl y la reevaluación de Gustafson discutida en la Sección 3, el dominio de SCLI favorece conceptualmente la lectura de Gustafson: el volumen de reservas que el sistema procesará en producción crecerá con el número de periodos académicos acumulados y con la adopción del sistema por más laboratorios, en lugar de permanecer fijo en las 10^5 filas usadas como conjunto de referencia en esta entrega. Sin mediciones de $t(N)$ para al menos dos valores de N , sin embargo, ninguna de las dos leyes puede aplicarse más allá de esta discusión cualitativa; ajustar p o α empíricamente es, junto con la ejecución del protocolo comparativo, el primer paso pendiente hacia la E4.

8.2. Amenazas a la validez

Internas. (i) Ausencia de repeticiones en la medición de latencia de tolerancia a fallos: una sola observación por escenario no permite distinguir variación real de ruido del sistema anfitrión. (ii) Calentamiento de JVM no aislado: como el pipeline PySpark aún no se ejecutó, no hay evidencia propia sobre cuánto afecta el arranque de la máquina virtual de Java al tiempo total reportado por el orquestador cuando se use.

(iii) Estado compartido del equipo de desarrollo: las capturas se tomaron en máquinas de desarrollo individuales de los integrantes, no en un entorno de medición dedicado y aislado de otros procesos.

Externas. (i) Representatividad del dataset: las 100 000 filas cargadas se generaron de forma sintética y reproducible, pero no reflejan necesariamente la distribución real de reservas de laboratorios de la UTEQ en producción. (ii) Generalidad a otras cargas: los resultados de tolerancia a fallos se midieron sobre una única consulta de agregación (`SELECT COUNT(*)`); el comportamiento ante consultas transaccionales más complejas o con mayor concurrencia de escritura no se evaluó. (iii) Generalidad de la topología: los tres nodos del clúster E3 se ejecutan como contenedores en un mismo equipo físico, sin la latencia de red ni la heterogeneidad de hardware que tendría un despliegue geo-distribuido real como el que describen Taft *et al.* [6] para clústeres de producción; los resultados de latencia de este informe no deben extrapolarse directamente a un escenario multi-región.

Validez de constructo. La Tabla 2 usa la latencia de una única consulta como indicador de disponibilidad, lo cual es una operacionalización razonable pero parcial del concepto de “tolerancia a fallos”: un indicador más completo –pendiente para la E4– combinaría esta métrica con la tasa de reintentos de transacciones (`crdb_transaction_retries_total`, Módulo G) y con el tiempo de reconvergencia del líder Raft tras una elección, ninguno de los cuales se midió en este ciclo.

8.3. Limitaciones

Las limitaciones principales de esta entrega, declaradas explícitamente en lugar de disimuladas, son las siguientes.

Fragmentación no materializada. La fragmentación temporal del ADR-003 está diseñada y justificada, pero el DDL correspondiente no está en `db/schema.sql`; en consecuencia, ninguna de las afirmaciones de la Sección 4 sobre poda de particiones se verificó todavía con `EXPLAIN` sobre el esquema real.

Comparación de rendimiento no ejecutada. La comparación pandas/PySpark no tiene ejecuciones registradas y, en consecuencia, no hay verificación empírica de la Ley de Amdahl ni de la reevaluación de Gustafson sobre datos propios del proyecto; toda cifra de *speedup* que se mencionara sin esta ejecución sería una invención, no un resultado.

Observabilidad incompleta. Las métricas Prometheus incrementales exigidas por el Módulo G de la guía de la entrega (`crdb_query_duration_seconds`, `crdb_transaction_retries_total` y `crdb_pool_active_connections`) no se implementaron en este ciclo; el `endpoint /metrics` heredado de la entrega anterior no expone todavía indicadores específicos de las consultas al clúster distribuido.

Alcance de la refactorización. Solo el microservicio de reservas y solicitudes consume el clúster distribuido; los otros tres microservicios permanecen sobre instancias de un solo nodo, de modo que las garantías de tolerancia a fallos descri-

tas en este informe no se extienden todavía al resto del sistema.

Escala del experimento de tolerancia a fallos. La verificación de la Sección 5.7 se realizó sobre una única consulta de conteo, sin concurrencia de escrituras simultáneas; el comportamiento del clúster bajo carga transaccional concurrente durante la caída de un nodo –que es plausiblemente distinto del caso de una sola conexión de lectura– no fue evaluado.

9. Conclusiones e integración con la Entrega 4

Esta entrega consolidó la capa de persistencia distribuida del sistema SCLI: el clúster CockroachDB de tres nodos está desplegado, verificado y demuestra tolerancia a la caída de un nodo con el comportamiento exacto que predice la teoría de consenso Raft, y la estrategia de fragmentación y replicación quedó documentada mediante dos ADR que fundamentan las decisiones de diseño en el patrón de acceso real del dominio. El pipeline analítico paralelo y su línea base secuencial están implementados y son funcionalmente equivalentes, aunque su comparación empírica de rendimiento queda pendiente. En conjunto, estos avances responden a los objetivos específicos 1 a 3 de la Sección 1 (diseño del esquema, despliegue del clúster y verificación de tolerancia a fallos) de manera completa, y a los objetivos 4 a 6 (protocolo comparativo, pipeline PySpark y verificación de las leyes de escalamiento) de manera parcial: el código y el protocolo existen, pero la ejecución y el análisis estadístico todavía no.

Respondiendo directamente a la pregunta de integración del Anexo B de la guía de la entrega –qué artefactos de esta entrega serán consumidos, sin modificación, por la Entrega 4– los artefactos que la Entrega 4 reutilizará directamente, sin modificación, son: el clúster CockroachDB E3 y su esquema de base de datos; los ADR-003 y ADR-004 como fundamento de cualquier decisión de observabilidad o escalamiento posterior; el pipeline `spark/pipeline.py` y su línea base `spark/baseline.py`; y el orquestador experimental `experiments/run_comparison.py`, listo para producir las mediciones que este informe no pudo reportar todavía. El trabajo inmediato hacia la E4 consiste en materializar el DDL de fragmentación, ejecutar el protocolo comparativo con las diez repeticiones especificadas, y extender el `endpoint` de métricas con los tres instrumentos Prometheus definidos en el Módulo G, sobre los cuales se construirá el panel de observabilidad consolidado de la siguiente entrega.

9.1. Ruta concreta hacia la Entrega 4

En orden de prioridad, el equipo propone la siguiente secuencia de trabajo para el inicio del próximo ciclo, cada paso condicionado a la finalización del anterior: (i) escribir y versionar el DDL de `PARTITION BY RANGE` con los límites trimestrales y las particiones de cobertura del ADR-003, y verificar con `EXPLAIN` que las consultas de disponibilidad usan poda de particiones; (ii) ejecutar `experiments/run_comparison.py` con $r = 10$ repeticiones por tratamiento sobre el dataset de 100 000 fi-

las ya cargado, alternando el orden de `pandas-baseline` y `pyspark-pipeline` conforme al protocolo de la Sección 7; (iii) calcular la media, la desviación típica y el intervalo de confianza al 95 % de las mediciones resultantes, y ajustar empíricamente p en la Ecuación (2); (iv) repetir la verificación de tolerancia a fallos de la Sección 5.7 con $r = 10$ repeticiones por escenario, en lugar de la observación puntual reportada en este informe; y (v) instrumentar los tres contadores e histogramas Prometheus del Módulo G sobre el microservicio de reservas, como base directa del panel de observabilidad que la E4 exige para el sistema completo.

Referencias

- [1] A. Venčkauskas, D. Kukta, Š. Grigaliūnas, and R. Brūzgienė, “Enhancing microservices security with token-based access control method,” *Sensors*, vol. 23, Mar. 2023. Art. 3363. doi: 10.3390/s23063363.
- [2] S. Li, H. Zhang, Z. Jia, C. Zhong, C. Zhang, Z. Shan, J. Shen, and M. A. Babar, “Understanding and addressing quality attributes of microservices architecture: A systematic literature review,” *Inf. Softw. Technol.*, vol. 131, Mar. 2021. Art. 106449. doi: 10.1016/j.infsof.2020.106449.
- [3] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Cham, Switzerland: Springer, 4th ed., 2020. doi: 10.1007/978-3-030-26253-2.
- [4] M. Kleppmann and P. Alvaro, “Research for practice: Convergence,” *Commun. ACM*, vol. 65, pp. 104–106, Nov. 2022. doi: 10.1145/3563901.
- [5] E. A. Lee, R. Akella, S. Bateni, S. Lin, M. Lohstroh, and C. Menard, “Consistency vs. availability in distributed cyber-physical systems,” *ACM Trans. Embed. Comput. Syst.*, vol. 22, pp. 1–24, Sept. 2023. Art. 138. doi: 10.1145/3609119.
- [6] R. Taft *et al.*, “CockroachDB: The resilient geo-distributed SQL database,” in *Proc. 2020 ACM SIGMOD Int. Conf. Management of Data*, (Portland, OR, USA), pp. 1493–1509, 2020. doi: 10.1145/3318464.3386134.
- [7] D. Huang *et al.*, “TiDB: A Raft-based HTAP database,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3072–3084, 2020. doi: 10.14778/3415478.3415535.
- [8] Z. Wang, T.-H. P. Chen, H. Zhang, and S. Wang, “An empirical study on the challenges that developers encounter when developing Apache Spark applications,” *J. Syst. Softw.*, vol. 194, Dec. 2022. Art. 111488. doi: 10.1016/j.jss.2022.111488.
- [9] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song, “Threats to validity in software engineering research: A critical reflection,” *Inf. Softw. Technol.*, vol. 164, Dec. 2023. Art. 107329. doi: 10.1016/j.infsof.2023.107329.
- [10] S. Baltes and P. Ralph, “Sampling in software engineering research: A critical review and guidelines,” *Empir. Softw. Eng.*, vol. 27, no. 4, 2022. Art. 94. doi: 10.1007/s10664-021-10072-8.
- [11] T. Taipalus, “Database management system performance comparisons: A systematic literature review,” *J. Syst. Softw.*, vol. 208, Feb. 2024. Art. 111872. doi: 10.1016/j.jss.2023.111872.
- [12] C. Poolla and R. Saxena, “On extending Amdahl’s law to learn computer performance,” *Microprocessors and Microsystems*, vol. 96, Feb. 2023. Art. 104745. doi: 10.1016/j.micpro.2022.104745.
- [13] X.-H. Sun and X. Lu, “The memory-bounded speedup model and its impacts in computing,” *J. Comput. Sci. Technol.*, vol. 38, pp. 64–79, Jan. 2023. doi: 10.1007/s11390-022-2911-1.